

Examen L2 info d'administration système - corrigé

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

I - Chemins et permissions

1. Vous vous trouvez dans le répertoire `/home/user/jeux/`. Quel est le chemin relatif le plus court vers le fichier `/home/user/travail/sysadmin.txt` ?

Réponse : `../travail/sysadmin.txt`

Mauvaises réponses rencontrées :

- `"cd ../travail/sysadmin.txt"`. La question demande un chemin, pas une commande. Par ailleurs, faire un `cd` avec comme argument un chemin de ce qui est probablement un fichier régulier n'a pas de sens, car `cd` prend comme argument le chemin d'un répertoire.
- `"~/travail/sysadmin.txt"`. En supposant que votre home est `/home/user/`, ce chemin est correct mais il est absolu, il ne dépend pas du répertoire courant.
- `"../travail/sysadmin.txt"`. Ce chemin (absolu) part de la racine et est équivalent à `/travail/sysadmin.txt` (le répertoire parent de la racine est la racine elle-même).

2. Depuis le même répertoire `/home/user/jeux/`, donnez un autre chemin relatif vers le fichier `/home/user/travail/sysadmin.txt`.

Réponse : `../../user/travail/sysadmin.txt`

Autre réponse valable (exemple) : `../../travail/../../user/jeux/../../travail/./sysadmin.txt`

Commentaire : dans un arbre, il n'y a un et un seul plus court chemin entre deux noeuds (faites un dessin pour vous en convaincre). Ainsi, il faut forcément faire des détours inutiles pour répondre à cette question. Les liens symboliques peuvent parfois créer des raccourcis (cf le jeu Find your path <http://demo710.univ-lyon1.fr/FYP/>).

3. Donnez une commande qui donne à tous les users du système le droit d'exécution sur le fichier `script.sh` qui se trouve dans le répertoire courant, sans modifier les autres permissions :

Commande : `chmod a+x script.sh`

Autre commande valable : `chmod ugo+x script.sh`

Mauvaises réponses rencontrées :

- `"chmod 111 script.sh"`. Cette commande donne les droits d'exécution à tous les users du système mais efface les autres permissions. Après cette commande, tout le monde aura perdu les droits d'écriture et de lecture.
- `"chmod u+x script.sh"`. Cette commande rajoute le droit d'exécution uniquement à l'utilisateur propriétaire.
- `"chmod +x script.sh"`. Le résultat de cette opération dépend de l'umask (pas vu en cours), à éviter si on sait pas ce que c'est.
- `"chmod o+x script.sh"`. Le `o` correspond à l'ensemble des users qui ne sont pas l'utilisateur propriétaire et qui ne font pas partie du groupe propriétaire du fichier. Il manque donc du monde.

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

4. Donnez une commande qui empêche tous les users autres que l'utilisateur propriétaire (vous) de modifier le fichier `plop.txt`, sans modifier les autres permissions :

Commande : `chmod go-w plop.txt`

Autre commande possible : `chmod g-w,o-w plop.txt`

Mauvaises réponses rencontrées :

- "`chmod o-w plop.txt`" Les membres du groupe propriétaire autres que l'utilisateur propriétaire ne sont pas pris en compte par `o`.

5. Donnez une commande qui supprime toutes les permissions des fichiers du répertoire courant et de sa sous-arborescence.

Commande : `sudo chmod -R 0 .`

Remarque : l'option `-R` (pour "recursive") est utilisée dans de nombreuses commandes pour itérer dans les sous-répertoires du répertoire spécifié.

Remarque : `chmod -R 0 .` ne marche pas pour la raison suivante : le parcours de la sous-arborescence avec l'option `-R` commence par le répertoire courant `.`, de sorte qu'une fois que ce répertoire a perdu les droits en lecture et exécution, il est impossible à la commande de parcourir la sous-arborescence pour enlever les autres permissions ! Comme `root` a tous les droits, la commande `sudo chmod -R 0 .` va enlever les permissions pour tous les fichiers de la sous-arborescence jusqu'au bout. Pour le partiel, cette subtilité n'a pas été prise en compte dans la notation, `chmod -R 0 .` est accepté. Une personne qui aurait pensé à la commande qui se tire dans les pieds aurait eu un sacré bonus. Devoir maison : essayez chez vous pour vous en convaincre !

Remarque : la commande `sudo chmod -R 000 .` est valide mais l'utilisation de `000` est inutile, car le nombre est interprété comme un nombre en octal (base 8). Notez au passage que les permissions sont codées sur 12 bits et pas 9 car il manque les permissions `setuid` `setgid` et `sticky bit` qu'on a pas eu le temps d'aborder en cours (cf slides), ainsi il les permissions sont en fait écrites avec 4 caractères octals. Comme ce sont les 3 bits de poids forts et qu'ils sont la plupart du temps désactivés, ce zéro est souvent omis, mais il arrive dans certaines docs de voir des permissions explicites comme `0644` par exemple (ce nombre écrit en octal étant égal à `644`).

6. Sans tenir compte des droits existants, faites en sorte que pour le fichier `./bidule.sh`,

- l'utilisateur propriétaire (vous) puissiez lire, écrire, exécuter,
- les membres du groupe possédant le fichier puissent lire, exécuter mais pas écrire,
- les autres ne puissent rien faire.

Commande : `chmod 750 bidule.sh`

Remarque : ici, on veut faire table rase du passé, ne pas tenir compte des permissions existantes, donc tout doit être spécifié explicitement, sans rien oublier. L'utilisation de l'octal est recommandé pour ça, et est très lisible.

Les commandes suivantes ont été acceptées car elles écrasent bien les 9 bits de permissions vues en cours. Mais elles ne modifient pas les 3 bits de permissions `setuid` `setgid` et `sticky bit` qu'on a pas eu le temps d'aborder en cours (cf slides) :

- `chmod u=rwx,g=rx,o= bidule.sh`
- `chmod u=rwx,g=rx,o-rwx bidule.sh`
- `chmod u+rwx,g+rx-w,o-rwx bidule.sh`

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

II - Analyse de commande ls

Votre nom d'utilisateur est `toto` et la commande `ls -ilah` dans le répertoire courant donne le résultat suivant :

```
45672 drwxr-xr-x 10 toto toto 4.0K mai 14 01:16 .
4471 drwxr-xr-x 30 root root 4.0K mars 29 18:11 ..
1526 -rw-r--r-- 1 toto toto 3,5K oct. 16 2019 .bashrc
1743 drwx----- 2 toto toto 4,0K mars 24 04:19 .ssh
1233 -rw-r--r-- 1 toto toto 6 mai 24 16:10 'a -> aze'
4887 -rw-r--r-- 1 tata tata 6 mai 24 16:10 aze
5889 drwxr-xr-x 2 toto toto 4,0K mai 24 22:23 Bureau
10000 -r-xr-x--- 2 root root 14K mars 25 20:41 config
10000 -rwx----- 2 root root 14K mars 25 20:41 configuration
6636 lrwxrwxrwx 1 toto bibi 6 mai 24 22:24 Desktop -> Bureau
7234 -rw-r--r-- 3 toto toto 337M mai 26 17:34 netinst.iso
8052 lrwxrwxrwx 1 toto toto 1 mai 25 23:55 racine -> /
9303 brw-rw---- 1 root disk 8, 1 mai 19 09:19 sda1
```

1. À quel groupe appartient le fichier `config` ?

Réponse : `root` (se lit sur la 5e colonne)

2. Combien de liens symboliques se trouvent dans le répertoire courant ?

Réponse : 2

3. À quoi reconnaissez-vous un lien symbolique ?

Réponse : le premier caractère de la 2e colonne (indiquant le type de fichiers) doit être un `l`

Mauvaises réponses rencontrées :

- compter les flèches `->` dans la dernière colonne. En effet, rien n'empêche un nom de fichier de contenir cette chaîne de caractères. C'est par exemple le cas du fichier régulier nommé `'a -> aze'`.

4. Quelle est, approximativement, la taille utilisée sur le disque par les fichiers réguliers qui se trouvent dans ce répertoire ? Expliquez.

Réponse : `337M` (c'est à dire `337` Mebi-octets, soit $337 \times 1024 \times 1024 = 337 \times 2^{20}$ octets, soit $337 \times 2^{20} \times 8 = 337 \times 2^{23}$ bits). Le fichier `netinst.iso` est largement plus gros que tous les autres dont la taille est négligeable (quelques kilos).

Remarque : on pourrait être tenté-e de faire la somme malgré tout, mais ajouter `14K` à `337M` n'aurait pas de sens car il faut comprendre que `337M` est un arrondi, la taille réelle du fichier pourrait, par exemple, être `337.4M` et les `400K` supplémentaires qui sont arrondis sont bien supérieurs aux `14K` du 2e plus gros fichier.

Mauvaises réponses rencontrées :

- "4.0K qu'on peut lire sur la première ligne". Il s'agit de la taille prise par le contenu du répertoire, c'est à dire la taille prise par la liste des paires (`nom`, `inode`) représentant les fichiers du répertoire.

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

5. Quel fichier ne semble pas à sa place dans ce répertoire ? Pourquoi ? Dans quel répertoire devrait-il se trouver ?

Réponse : le fichier `sda1` est un block device (type `b`), il ne peut se trouver que dans le répertoire `/dev` sur lequel est monté un système de fichier spécial de type `udev`.

Mauvaises réponses rencontrées :

- "`config` devrait se trouver dans le répertoire `.ssh`". Il n'y a pas que SSH qui appelle son fichier de configuration `config`, et l'utilisateur peut très bien appeler son fichier de recettes de cuisine `config`.
- "le fichier `aze` car son utilisateur propriétaire est `tata`". Rien ne l'interdit (on peut, par exemple, imaginer que ce fichier a été mis là par `root` pour permettre à `tata` d'écrire des choses que `toto` pourra facilement lire dans son home).

6. Pouvez-vous effacer le fichier `config` ? Pourquoi ?

Réponse : oui. Le répertoire `.` qui contient ce fichier appartient à l'utilisateur `toto` qui a les droits d'écriture.

Information complémentaire : il faut comprendre qu'effacer un fichier ne modifie pas le contenu du fichier. Ce qui est modifié c'est le répertoire : on y enlève le nom du fichier correspondant et son numéro d'inode. J'insiste sur ce fait : il est important de comprendre un minimum comment fonctionnent les choses pour ne pas se tromper là-dessus. Imaginez les trous de sécurité qui pourraient être causés par une mauvaise compréhension du fonctionnement des systèmes de fichiers ! Voir la démo "Signification des permissions (r,w,x) pour les répertoires" sur la page du cours.

Devoir maison : essayez chez vous pour vous en convaincre !

Mauvaises réponses rencontrées :

- "non parce qu'on a pas les droits en écriture". Effacer n'est pas la même chose que modifier le contenu, effacer c'est faire un `unlink` : on modifie le répertoire (en supprimant l'entrée qui correspond au nom de fichier), on décrémente le nombre de liens dans l'inode du fichier, si ce nombre vaut zéro on déclare cet inode inutilisé dans la table des inodes, mais on ne touche pas à son contenu. Je ne sais pas si c'est une bonne comparaison, mais pour avoir le droit de créer un fichier, il faut pouvoir écrire dans le répertoire, les permissions du fichier (qui n'existe pas encore) n'entrent pas en jeu.
- "on ne peut pas effacer un fichier si on a pas les droits d'exécution sur le fichier". Le terme "exécuter" n'a ici rien à voir avec la mise à mort.

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

7. Vous voulez libérer de l'espace sur votre disque. Combien d'espace gagnerez-vous avec la commande `rm netinst.iso` ? Pourquoi ?

Réponse : aucun espace ne sera libéré. Le fichier nommé `netinst.iso` possède 3 liens, ce qui signifie que ce fichier dont le numéro d'inode est 7234 apparaît sous trois noms dans l'arborescence (une fois dans le répertoire courant et 2 fois ailleurs). Ainsi, supprimer ce nom ne va pas libérer d'espace puisque le contenu du fichier doit rester disponible (on doit pouvoir accéder au contenu du fichier qui apparaît sous deux autres noms).

Mauvaises réponses :

- "on gagne 337 M"
- "on gagne $3 * 337$ M" (pourquoi ?)
- "il ne faut pas supprimer un `.iso`". Comme expliqué dans le cours, l'extension d'un fichier n'est qu'une partie de son nom et n'a que peu d'intérêt pour le système, on peut tout à fait appeler `machin.iso` un fichier au format pdf et `bidule.pdf` un fichier au format png. Ici `netinst.iso` est probablement une image d'installation de Debian par le réseau qu'il faut copier sur une clef USB ou utiliser pour fabriquer une VM virtualbox (c'est cohérent avec sa taille).
- "le répertoire apparaît 3 fois dans le répertoire courant". Le fichier dont le numéro d'inode est 7234 apparaît une seule fois dans le répertoire courant, sous le nom `netinst.iso`. Il apparaît bien 3 fois dans l'arborescence, mais les 2 autres fois c'est ailleurs, dans d'autres répertoires.

8. Combien de sous-répertoires contient le répertoire parent du répertoire courant ? Pourquoi ?

Réponse : 28. Le répertoire parent du répertoire courant est nommé `..`. Le nombre de liens vers le répertoire parent du répertoire courant est 30. Le répertoire parent du répertoire courant apparaît :

- une fois dans son répertoire parent (avec son nom)
- une fois dans lui-même (en tant que `.`)
- une fois dans chacun de ses sous-répertoires (en tant que `..`)

$$30 - 2 = 28$$

Remarque: c'est une variante d'une question du qcm d'entraînement sur les attributs d'inode.

9. Quel fichier régulier apparaît avec deux noms différents dans le répertoire courant ? Pourquoi ?

Réponse : le fichier dont le numéro d'inode est 10000 apparaît sous le nom `config` et sous le nom `configuration`.

Remarque : plusieurs étudiant-es pensent que l'un des 2 noms est un lien physique vers l'autre. En fait la relation entre les 2 noms est parfaitement symétrique : ces deux noms ont un lien physique vers un inode.

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

10. Deux incohérences se sont glissées dans le résultat de la commande, lesquelles ?

Incohérence 1 : les fichiers nommés **config** et **configuration** sont le même fichier (même inode) et pourtant ils n'ont pas les mêmes permissions. C'est impossible car les permissions sont inscrits dans l'inode.

Incohérence 2 : Le nombre de liens du répertoire courant (nommé **.**) est 10. Comme le répertoire courant n'a que 2 sous-répertoires, son nombre de liens devrait être 4 (il apparaît une fois dans son répertoire parent sous un certain nom, une fois dans lui-même sous le nom **.** et une fois dans chaque sous-répertoire sous le nom **..**).

Mauvaises réponses rencontrées :

- "la date du fichier **.bashrc** (**oct. 16 2019**)". Cette date est valide : les autres dates ne précisent pas l'année car ce sont des fichiers modifiés en 2021 donc on économise de la place pour pouvoir préciser l'heure dans le même espace.
- "la taille **8,1** du fichier **sda1**". Ces deux nombres (appelés majeur et mineur) séparés par une virgule sont spécifiques aux devices, ils ne correspondent pas à leur taille, mais permettent d'identifier le driver à utiliser (cf slides du cours).
- "un même fichier ne peut pas apparaître deux fois dans un répertoire". Si (sous des noms différents).
- "le lien symbolique **Desktop -> Bureau** n'est valable". **Bureau** est un chemin (relatif) valable donc le lien symbolique est valable. Par ailleurs, le fichier **Bureau** existe bien, donc le lien n'est pas cassé.
- "le lien **racine -> /** est un lien mort". D'une part les liens morts ça existe. D'autre part, ce lien n'est pas un lien mort car le chemin **/** existe.
- "la taille du répertoire (**4K**) est inférieure à la somme des tailles de ses fichiers". Rien d'anormal, la taille du répertoire est la taille de son contenu, c'est à dire la liste des paires (**nom, inode**).
- "**netinst.iso** apparaît 3 fois dans le répertoire courant". Il n'apparaît qu'une fois (et d'ailleurs les noms de fichiers d'un répertoire sont uniques).
- "les tailles des fichiers n'ont pas la même unité malgré l'option **-h**". L'option **-h** permet justement d'adapter l'unité à l'ordre de grandeur de la taille des fichiers. Sans l'option **-h**, la taille de tous les fichiers est exprimée en octets (par défaut).
- "certaines tailles de fichier sont en **K** ou **M**, d'autres n'ont pas d'unité". Toutes les tailles de fichiers sont en octet, les lettres **K**, **M**, **G**, **T** sont des facteurs multiplicatifs.
- "le répertoire parent **..** est affiché dans le répertoire courant". C'est normal, et ça permet de définir des chemins relatifs qui remontent dans l'arborescence (cf 1ère question du partiel).

Remarque : la présence de **sda1** est en effet une incohérence, donc ça n'est pas une mauvaise réponse, mais on en a déjà parlé dans la question II.5.

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

III - Filesystems et block devices

1. Que fait la commande

```
$ dd if=/dev/zero of=/tmp/image.iso bs=1MiB count=50
```

Réponse : elle crée un fichier `/tmp/image.iso` de taille 50 MiB rempli de zéros.

2. Quelle commande permet de lister les block devices présents sur le système ?

Commande : `lsblk`

3. Cette suite de commandes s'effectue sans erreur.

```
# rm -rf /opt/*
# rm -rf /mnt/*
# echo hop > /opt/ff
# echo hop > /mnt/gg
# mount -t ext4 /dev/sdf2 /mnt
# rm -rf /mnt/*
# echo hop > /mnt/hh
# umount /mnt
# mount -t ext4 /dev/sdf2 /opt
```

Juste après ces commandes, qu'affiche la commande

```
# ls /mnt
```

Réponse : `gg`

Qu'affiche la commande

```
# ls /opt
```

Réponse : `hh`

Explication : une explication illustrée sera peut-être ajoutée dans une version ultérieure de cette correction (et on en parlera lors des révisions).

4. Étant donné un fichier régulier, dans quel ordre doit-on effectuer les opérations suivantes :

- (a) Créer une partition
- (b) Détacher le block device
- (c) Démonter le "bidule"
- (d) Monter le "bidule" sur un répertoire existant de l'arborescence
- (e) Créer un nouveau fichier sur ce système de fichiers
- (f) Créer une table de partitions
- (g) Promouvoir le fichier en block device
- (h) Installer un système de fichiers

Réponse : `g f a h d e c b`

Commentaire : voir le cours, les demos sur les block devices, et la feuille "Image mentale sur les fichiers". Ici "bidule" signifiait "système de fichiers".

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

IV - Redirections

Une liste de commandes de manipulation de texte est donnée à la dernière page du sujet.

1. Un fichier texte nommé `nombres.txt` contient un entier par ligne, par exemple :

```
123
43
16643212332
14
```

Donnez une combinaison de commandes qui écrit dans un fichier nommé `petits.txt` les 5 plus petits nombres apparaissant dans le fichier `nombres.txt`

Commande : `cat nombres.txt | sort -n | head -n 5 > petits.txt`

Autre commande valable : `sort -n < nombres.txt | head -n 5 > petits.txt`

Autre commande valable : `sort -n nombres.txt | head -n 5 > petits.txt`

Remarque : la première commande n'est pas recommandée officiellement (UUOC), mais elle a l'avantage de rendre l'enchaînement plus clair (lecture de gauche à droite).

2. On souhaite rendre un texte lisible sur les écrans étroits. Sachant que :

- la commande `tr a-z A-Z` remplace les minuscules du texte de l'entrée standard en majuscules et l'envoie dans la sortie standard,
- la commande `fold -w 30 -s` réorganise le texte de l'entrée standard de sorte à ce que chaque ligne ne fasse pas plus de 30 caractères et l'envoie dans la sortie standard,

faites en sorte que le contenu du fichier `lorem.txt` soit écrit en majuscules et ait des lignes d'au plus 30 caractères dans le fichier `lorem.lisible`

Commande : `cat lorem.txt | tr a-z A-Z | fold -w 30 -s > lorem.lisible`

Mauvaises réponses :

- `"echo lorem.txt | tr a-z A-Z | fold -w 30 -s > lorem.lisible"`. Cette commande écrit la chaîne `"LOREM.TXT"` dans le fichier `lorem.lisible` car `echo lorem.txt` envoie la chaîne de caractère `"lorem.txt"` dans la sortie standard et pas le contenu du fichier `lorem.txt`.

3. Pour une application sur smartphone, vous devez transformer tous les fichiers dont l'extension est `.txt` présents dans une sous-arborescence en fichiers lisibles comme dans la question précédente. Comment vous y prendriez-vous (réponse approximative acceptée) ?

Réponse : on utilise `find` pour itérer sur les fichiers de la sous-arborescence.

Commande possible (non exigé pour valider) :

```
find -type f -name '*.txt' -exec sh -c 'cat {} | tr a-z A-Z | fold -w 30 -s > {}.lisible' \;
```

4. Un fichier de 100 lignes de texte nommé `fichier.txt` se trouve dans le répertoire courant. Que retourne la commande suivante

```
cat fichier.txt | wc -l | wc -l
```

Réponse : 1

Explication : `cat fichier.txt` envoie le contenu du fichier `fichier.txt` dans la sortie standard. Comme ce fichier fait 100 lignes, la commande `cat fichier.txt | wc -l` envoie la chaîne de caractères 100 dans la sortie standard. Comme la chaîne 100 ne fait qu'une ligne, la commande `cat fichier.txt | wc -l | wc -l` va envoyer la chaîne 1 dans la sortie standard.

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

V - Crypto

1. Comment s'est-on assuré que vous seul-e pouvez vous connecter en tant que root sur votre conteneur par SSH ? Décrivez la procédure.

Réponse :

- l'étudiant-e génère une paire de clefs avec la commande `ssh-keygen`
- l'étudiant-e envoie la clef publique à l'enseignant en utilisant son adresse mail universitaire.
- l'enseignant ajout la clef publique de l'étudiant-e au fichier `/root/.ssh/authorized_keys`
- ainsi, lorsqu'une personne demande au serveur SSH de se connecter en tant que `root`, le serveur SSH du conteneur challenge ce client pour vérifier qu'il dispose de la clef privée correspondant à l'une des clefs publiques copiées dans `/root/.ssh/authorized_keys`. L'étudiant-e qui possède la clef privée réussit à s'identifier, les autres personnes, non.

Remarque : c'est pour cela qu'il ne faut **jamais** partager sa clef privée.

Remarque : contrairement à une authentification par mot de passe, la clef privée du client SSH n'est pas envoyée au serveur pour vérification.

2. Comment s'est-on assuré que lorsque vous faites un SSH vers votre conteneur, vous soyez sur-e que vous vous connectez bien à votre conteneur et pas à une machine malveillante qui se trouverait sur la route. Décrivez la procédure.

Réponse :

- l'enseignant envoie à l'étudiant-e par mail le fingerprint de la clef publique du serveur SSH de son conteneur.
- lors de la première connexion de l'étudiant-e à son conteneur, l'étudiant-e vérifie que le fingerprint est le bon et, en acceptant, ce fingerprint est ajouté à son fichier `~/.ssh/known_hosts` pour lui éviter de le revalider à chaque connexion. Si jamais la clef publique du serveur SSH change, vu que son fingerprint est différent de celui enregistré dans le fichier `~/.ssh/known_hosts`, le client SSH refusera de se connecter et demandera à l'user de valider le nouveau fingerprint.

Remarque : le fingerprint de la clef publique du serveur n'est autre qu'un hash de cette clef, ce qui permet de vérifier une chaîne de caractères plus courte que la clef elle-même.

3. Décrivez les fichiers qui sont impliqués dans les opérations précédentes, et donnez leur rôle.

Réponse :

Fichiers impliqués dans les opérations de la question 1,

- la clef privée de l'étudiant-e (coté client) : `~/.ssh/id_rsa`
- la clef publique l'étudiant-e (coté client) : `~/.ssh/id_rsa.pub`
- une copie de la clef publique de l'étudiant-e (coté serveur) : `/root/.ssh/authorized_keys`

Fichiers impliqués dans les opérations de la question 2,

- les clefs privées du serveur SSH : `/etc/ssh/ssh_host_*_key`
- les clefs publiques du serveur SSH : `/etc/ssh/ssh_host_*_key.pub`
- la liste des fingerprints des clefs publiques des serveurs SSH que l'étudiant-e a validé : `~/.ssh/known_hosts` (coté client)

Remarque : le serveur SSH a plusieurs paires de clefs qui utilisent plusieurs algorithmes de chiffrement différents (RSA, courbes elliptiques, ...), de sorte à pouvoir répondre à des clients SSH de diverses générations.

Réponse hors sujet : "le fichier `~/.ssh/config`". Ce fichier permet de définir des options selon les connexions pour éviter de les passer dans la commande `ssh`, il n'apparaît donc pas explicitement dans les deux questions précédentes. À la limite,

- pour la question 1, on pourrait argumenter qu'il est possible de définir des clefs autres que `~/.ssh/id_rsa` dans ce fichier avec l'option `IdentityFile`,
- pour la question 2, on pourrait argumenter sur le fait que cela limite le risque de faute de frappe dans le nom de domaine de la machine qui héberge le serveur SSH et qu'un adversaire ait enregistré un nom de domaine très voisin.

VI - Réseau

1. Que trouve-t-on dans le fichier `/etc/services` ?

Réponse : une liste des services réseau internet, fournissant une correspondance entre des noms textuels faciles à mémoriser pour les services (par exemple `ssh`, `http`, `https`) et les numéros de ports qui leur sont assignés (par exemple 22, 80, 443).

Pour plus de détails : `man 5 services`

2. À quoi sert la commande `socat` ?

Réponse possible : `socat` permet d'établir un flux de données bidirectionnel entre deux extrémités locales ou distantes, ces extrémités pouvant être des sockets (TCP, UDP, unix), des tubes nommés, des fichiers réguliers, des character devices (terminaux, ports séries), etc. Cette flexibilité permet par exemple (voir les exemples du man) :

- de simuler un client web (protocole HTTP à la main) ou mail (protocole SMTP)
- de construire des tunnels
- de

3. Dans quel répertoire se trouvent les fichiers de configuration du serveur web ?

Réponse : `/etc/nginx/`

Rappel : les fichiers de configuration du système se trouvent dans le répertoire `/etc`.

Mauvaises réponses rencontrées :

- `"/var/www/html/"`. Ce répertoire contient le contenu du site web, pas la configuration du serveur.

4. Quel enregistrement DNS permet d'associer une adresse IPv6 à un nom de domaine ?

Réponse : AAAA

Remarque : concernant le DNS, le mot "enregistrement" signifie "champ". En anglais, on dit "DNS record".

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).

VII - Sysadmin

1. Vous voulez servir une appli web dynamique avec `uwsgi`. Vous installez ce paquet sur votre machine puis vous prenez le train sans réseau. Comment faites-vous pour vous documenter et avancer dans sa configuration pendant votre trajet ?

Réponse :

- un `man -k uwsgi` pour voir quels manuels parlent de `uwsgi` ne fait pas de mal. On y trouvera une page sur la commande `uwsgi` (section 1 du manuel). Elle permet de lancer le serveur via la ligne de commande, ce qui peut être intéressant pour tester diverses options avant de configurer le serveur `uwsgi` (qui sera lancé automatiquement comme un daemon par `systemd`).
- jeter un oeil dans le répertoire de configuration `/etc/uwsgi` car il pourrait y avoir des exemples de configuration (comme les templates dans la conf de `nginx`). Dans ce cas particulier, il se trouve qu'un fichier `/etc/uwsgi/apps-enabled/README` nous permettra, de fil en aiguille, de découvrir les fichiers
- lister les fichiers qui ont été installés par le paquet `uwsgi` et regarder ceux qui

Remarque : avant de prendre le train, il peut être judicieux de faire quelques recherches sur le web et garder ouverts les onglets qui semblent intéressants.

2. Vous voulez ajouter un fichier `LICENCE.txt` à un dépôt `git` distant. Quel est le problème avec la suite de commandes suivantes ?

```
$ git clone super.netlib.re:depot.git
$ cd depot
$ echo "This software is released under the GPL 3.0" > LICENCE.txt
$ git commit -m "ajout d'un fichier de licence"
$ git push
```

Réponse : il n'y a rien à commiter puisque rien n'a été ajouté à la staging area. Il manque une commande `git add LICENCE.txt` avant la commande `git commit`.

Remarque : `git init` sert à initialiser dépôt `git` (à partir de rien), elle n'est pas utile quand on clone un dépôt existant (`git clone` crée un dépôt sur la machine locale).

Remarque : quand on clone un dépôt dont le nom finit par `.git`, cette extension n'apparaît pas dans le répertoire qui héberge le dépôt cloné.

3. Combien de valeurs distinctes peut-on coder sur 6 bits ?

Réponse : 2^6 soit 64.

Mauvaises réponses (incompréhensibles) : "32" (pourquoi ?), "63" (pourquoi ?)

4. Soit n le nombre écrit en binaire 100110100110. Donnez, en binaire, une approximation à la louche de $n^2 = n*n$.

Réponse : 1000000000000000000000 (22 zéros). En effet n s'écrit avec 12 bits donc il est de l'ordre de 2^{11} , donc n^2 est de l'ordre de 2^{22} .

Remarque : la valeur exacte est 10111010001011110100100 mais on ne vous demandait pas de la calculer.

Commandes de manipulation de texte

Ces commandes permettent de manipuler du texte et se combinent très bien entre elles.

- éditer un fichier texte : **vim** ou **emacs**
- envoyer du texte dans la sortie standard : **echo**
- envoyer le contenu d'un fichier dans la sortie standard, concaténer plusieurs fichiers : **cat**
- afficher le contenu d'un fichier texte et pouvoir y naviguer (cf cours introduction, raccourcis proches de **vim**) : **less**
- rechercher une chaîne de caractère dans un fichier : **grep** (options **-v**, **-c**, **-i**)
- afficher les différences entre deux fichiers texte : **diff** (voir aussi **vimdiff**)
- trier les lignes d'un fichier texte dans l'ordre alphabétique : **sort** (option **-n** pour tri numérique)
- éliminer les lignes consécutives répétées : **uniq**
- afficher les premières lignes d'un fichier texte : **head** (option **-n** pour spécifier le nombre de lignes)
- afficher les dernières lignes d'un fichier texte : **tail** (option **-n** pour spécifier le nombre de lignes)
- compter le nombre de lignes/mots/octets/caractères d'un fichier texte : **wc** (option **-l**)
- convertir des caractères : **tr**
- changer l'encodage d'un fichier texte : **iconv**
- couper chaque ligne de texte à une longueur donnée : **fold** (options **-s**, **-w**)
- substitution de motifs, suppressions de lignes : **sed**
- traitement de fichiers structurés en colonnes : **awk**
- afficher les portions imprimables d'un fichier binaire : **strings**
- afficher le contenu d'un fichier binaire en hexadécimal (2 octets par caractère) : **hexdump** (option **-C**)
- afficher le contenu d'un fichier binaire en octal (1 octet par caractère) : **od**

Note importante : l'auteur de ce sujet et de sa correction n'autorise pas sa reproduction, sous quelque format que ce soit, particulièrement sur les serveurs discord, les sites de revente d'annales, les chatbots des LLM (chatGPT, etc).